

გაკვეთილი 5 – SCSS
Math, Maps, Loops, @if
და Functions

უფრო ჭკვიანი SCSS – როცა კოდი თვითონ გვეხმარება
განმეორების შემცირებაში

FRONT-END განვითარება

SASS/SCSS

დღევანდელი 5 საათის რეკა

თეორია + ბევრი პატარა live კოდი + ერთი პროდუქტიული პრაქტიკული პროექტი

01

გახსნა

Variables, mixin, extend — მოკლე კავშირი
წინა გაკვეთილებთან

02

Math

ზომები, დაშორებები და მარტივი
გამოთვლები SCSS-ში

03

Maps + Loops

ფერების და ზომების ავტომატური
კლასები @each-ით

04

@if + Functions

პირობითი ლოგიკა და საკუთარი პატარა გამოთვლები

05

Mini Project

Badge / Card utility generator — @each, @if და @function ერთად

- ❶ მთავარი იდეა: SCSS-ში შეგვიძლია არა მხოლოდ style დავწეროთ, არამედ წესებიც შევქმნათ: რა გამოვრდეს, როდის შეიცვალოს და როგორ დაითვალოს.

სწრაფი კავშირი წინა გაკვეთილებთან

ახალი თემები ძველის გაგრძელებაა — უბრალოდ უფრო ავტომატური გზით

Variables

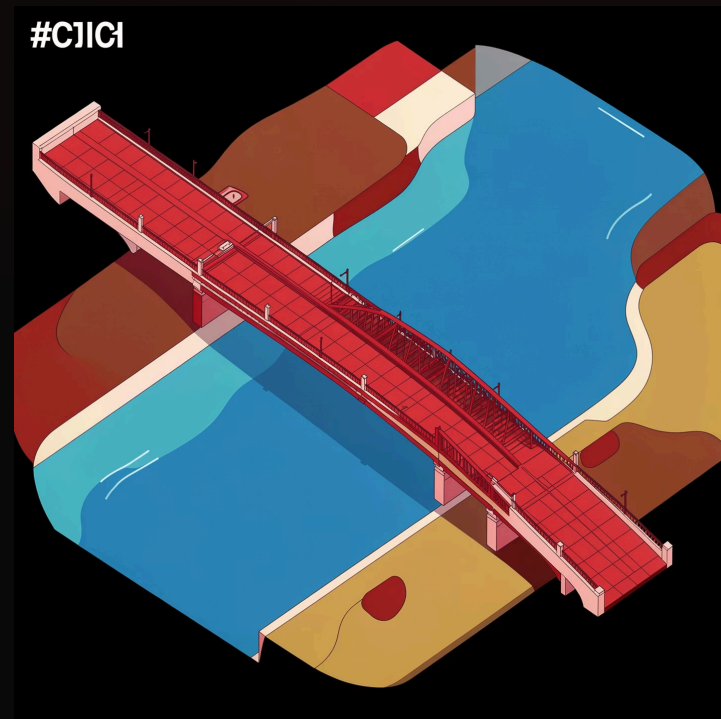
ერთ მნიშვნელობას
ვინახავთ: ფერი, ზომა,
დაშორება

Mixin

ერთ style block-ს
ბევრგან ვიყენებთ
@include-ით

Extend

საერთო base style-ს
ვაზიარებთ @extend-
ით



❑ დღევანდელი კითხვა: თუ ბევრი ფერი, ზომა ან კლასი გვაქვს, როგორ შევქმნათ ისინი უფრო სწრაფად და ნაკლები ხელით წერით?

SCSS Math – ზომების გამოთვლა პირდაპირ კოდში

რატომ **Math**?

SCSS-ში შეგვიძლია გამოვიყენოთ `+`, `-`, `*`, `/` ზომების დასათვლელად. ერთი საბაზისო მნიშვნელობიდან ყველა სხვა ზომა ავტომატურად ითვლება.

- კარგია `spacing`, `padding`, `font-size`-ისთვის
- მნიშვნელობა ერთგან იცვლება
- დიზაინი უფრო თანმიმდევრული გამოდის

Live Code მაგალითი

```
$base-space: 8px;

.card {
  padding: $base-space * 2;
  margin-bottom: $base-space * 3;
}
```

 თუ გვაქვს ერთი ძირითადი ზომა, დანარჩენი ზომები შეგვიძლია მისგან დავითვალოთ.

SCSS Maps – მნიშვნელობების შენახვა სახელებით

რატომ **Maps**?

Map ინახავს **key** → **value** წყვილებს. კარგია ფერებისთვის, ზომებისთვის და theme-ებისთვის. მნიშვნელობას ვიღებთ `map-get()`-ით.

- ყველა მსგავსი მნიშვნელობა ერთ ადგილას იყრის თავს
- ცვლილება მხოლოდ ერთ ადგილას
- კოდი უფრო ორგანიზებულია

📌 Map არის პატარა ლექსიკონი SCSS-ში: სახელს ვეუბნებით და ის შესაბამის მნიშვნელობას გვცდის.

Live Code მაგალითი

```
$theme-colors: (  
  info: #0984e3,  
  success: #00b894,  
  danger: #d63031  
);  
  
.info-title {  
  color: map-get($theme-colors, info);  
}
```


Loops – @each და @for

როცა ერთი და იგივე სტილი ბევრჯერ უნდა შეიქმნას, loop-ები ავტომატურად გვიქმნიან კლასებს.

@each – Map-თან ერთად

```
$colors: (  
  info: #0984e3,  
  success: #00b894  
);  
  
@each $name, $color in $colors {  
  .bg-#{$name} {  
    background-color: $color;  
  }  
}
```

`#{$name}` — Interpolation: ცვლადის სახელად ჩასმა class-ში.

@for – რიცხვებით

```
@for $i from 1 through 4 {  
  .m-#{$i} {  
    margin: $i * 8px;  
  }  
}  
  
// .m-1 = 8px  
// .m-4 = 32px
```

`through` — ბოლო რიცხვი შედის, `to` — არ შედის.

❓ საკლასო კითხვა: თუ map-ში დავამატებთ `danger` ფერს, რას შექმნის loop ავტომატურად?

@if – პირობითი ლოგიკა SCSS-ში

მაგალითი: ღია და მუქი ფონი

```
@function is-light($color) {  
  @if lightness($color) > 50% {  
    @return true;  
  } @else {  
    @return false;  
  }  
}  
  
.badge {  
  @if is-light($bg) {  
    color: #333;  
  } @else {  
    color: white;  
  }  
}
```

რატომ @if?

ზოგჯერ სტილი დამოკიდებულია პირობაზე. მაგალითად, თუ ფონი ღიაა — ტექსტი მუქი, თუ მუქია — ტექსტი ღია.

- კარგია თემებისთვის (light/dark)
- ზომის მიხედვით სტილის ცვლილება
- შეცდომების თავიდან აცილება

 **პირობა:** თუ ასეა — ეს სტილი, თუ არა — სხვა.

Functions – საკუთარი გამოთვლები

Function არის პატარა გამოთვლა, რომელსაც სახელს ვარქმევთ და ბევრგან ვიყენებთ. SCSS-ში უკვე არის ჩაშენებული ფუნქციები, მაგრამ შეგვიძლია საკუთარიც შევქმნათ.

რატომ **Functions**?

თუ ერთი და იგივე გამოთვლა ბევრგან გვჭირდება, function-ში შევინახავთ. შეცვლა მხოლოდ ერთ ადგილას.

მაგალითი: **Rem** კონვერტორი

```
@function to-rem($px) {  
  @return $px / 16px * 1rem;  
}  
  
.title {  
  font-size: to-rem(24px); //  
  1.5rem  
}
```

ჩაშენებული ფუნქციები

SCSS-ში არის `lightness()`, `darken()`, `mix()` და სხვა. შეგვიძლია გამოვიყენოთ `@if`-თან ერთად ფერების ლოგიკისთვის.

MINI PROJECT

Badge / Card Utility Generator

ყველაფერი ერთად: @each, @if და @function — პატარა, მაგრამ პროდუქტიული სისტემა



Map

ფერების და ზომების სია ერთ ადგილას



@each

ავტომატურად ქმნის .badge-info, .badge-success და ა.შ.



@if

ფერის მიხედვით ტექსტის ფერი იცვლება



@function

საკუთარი გამოთვლა ზომებისთვის ან ფერებისთვის



Made with GAMMA

დღევანდელი შეჯამება

1 Math

ერთი base value → დანარჩენი ზომები ავტომატურად

2 Maps

მნიშვნელობები სახელებით, map-get() გამოყენებით

3 Loops

@each და @for — ავტომატური კლასების შექმნა

4 @if + Functions

პირობითი ლოგიკა და საკუთარი გამოთვლები

- ✔ შემდეგი გაკვეთილი: BEM + SCSS ერთად — როგორ დავწეროთ სუფთა, ორგანიზებული და მასშტაბირებადი კოდი რეალურ პროექტში.

